

Wykrywanie włamań i ataków w sieciach komputerowych

Szymon Kluska, Kacper Krześciński

Abstract—Celem projektu jest opracowanie, implementacja, trening oraz ewaluacja modelu, który posłuży do wykrywania włamań i ataków w sieci komputerowej. Do jego realizacji zostanie wykorzystany model typu *Autoencoder*, uczony metodą *open-set recognition*. W ramach projektu zostanie wykonana również analiza i opracowanie uzyskanych wyników.

Index Terms—sieci komputerowe, ataki sieciowe, uczenie maszynowe, *Autoencoder*, *open-set recognition*

I. WSTĘP

INFRASTRUKTURA sieciowa jest stale narażona na różnego rodzaju zagrożenia. Analiza zebranych danych po wystąpieniu incydentów umożliwia specjalistom identyfikację metod włamań, a w konsekwencji - tworzenie skutecznych mechanizmów obronnych. Jednak atakujący nieustannie rozwijają swoje techniki i wprowadzają nowe formy nadużyć tak, aby pozostać niewykrywalnymi. Z tego powodu systemy oparte na uczeniu maszynowym muszą stale obserwować liczne wskaźniki telemetrii w czasie rzeczywistym, aby wychwytywać nawet niewielkie odchylenia i generować szybkie ostrzeżenia. Równie ważna jest jednak umiejętność rozpoznawania zupełnie nowych, wcześniej niespotykanych zagrożeń. Jest to kluczowe dla bezpieczeństwa sieci, aby system wczesnego ostrzegania zareagował nawet wtedy, kiedy z danym atakiem spotyka się po raz pierwszy.

II. OPIS OPRACOWYWANEJ KONCEPCJI

Zadaniem jest opracowanie, implementacja, trening oraz ewaluacja modelu, który posłuży do wykrywania włamań i ataków w sieci komputerowej. Model ten będzie uczony z wykorzystaniem metody *open-set recognition* [11], na danych etykietowanych jako "brak ataku". Celem będzie nauczenie modelu poprawne przypisywanie normalnego ruchu sieciowego do klasy "brak ataku". Natomiast jeśli jakaś próbka nie będzie pasować do tej klasy, model wykryje to jako anomalię, co należy interpretować jako potencjalny atak. Działający w ten sposób model, tzn. dopasowujący dane tylko do jednej klasy i wykrywający anomalie, nosi nazwę *Autoencoder* [10].

Trening, walidacja oraz testowanie modelu zostaną przeprowadzone na ogólnodostępnych danych dostępnych w Internecie [1, 2] i zostaną one przekształcone do postaci etykiety binarnej - atak vs brak ataku.

Celem projektu będzie odpowiedzenie na następujące pytania (*research questions*):

Q1) Czy modele typu *Autoencoder*, uczone metodą *open-set recognition*, poprawnie klasyfikują "normalny" ruch sieciowy?

Q2) Czy modele typu *Autoencoder*, uczone metodą *open-set recognition*, skutecznie wykrywają ogólnie znane, jak i nowe ataki sieciowe?

By odpowiedzieć na te pytania zostanie zaimplementowany model typu *Autoencoder*, a następnie zostaną przeprowadzone eksperymenty.

Ocena jakości predykcji zostanie wyznaczona za pomocą współczynnika F1 opisanego wzorem 1.

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (1)$$

Gdzie:

- TP (*True Positive*) - poprawnie wykryty atak.
- FP (*False Positive*) - fałszywy alarm.
- FN (*False Negative*) - przeoczony atak.
- TN (*True Negative*) - poprawnie rozpoznany normalny ruch.

Do implementacji modelu zostanie wykorzystany język Python [3] wraz z bibliotekami *scikit-learn* [4] i *PyTorch* [5]. Do obróbki danych i ich reprezentacji posłużą biblioteki *pandas* [6], *NumPy* [7] oraz *matplotlib* [8].

III. ALGORYTMY

LSTM *Autoencoder* to sieć neuronowa złożona z pary enkoder-dekoder. Enkoder odczytuje sekwencję wejściową oraz transformuje dane wejściowe a dekoder odpowiada za rekonstrukcję oryginalnej sekwencji. W przypadku normalnego ruchu sieciowego, błąd rekonstrukcji będzie znikomy, a w przypadku anomalii, znaczny [10].

Kluczową kwestią jest ustalenie progu dla błędu rekonstrukcji (*threshold*), od którego daną próbkę będziemy uznawać za atak. Próg ten należy wyliczyć na danych treningowych, czyli na czystym ruchu normalnym. W tym celu należy przepuścić wszystkie pakiety treningowe przez nauczonego model i dla każdego pakietu wyliczyć błąd rekonstrukcji. Następnie błędy te ustawia się w kolejności od najmniejszego do największego i podejmuje decyzję o ustaleniu progu. Np. w przypadku zdecydowania się na percentyl równy 90, algorytm idzie do miejsca, gdzie kończy się 90% najmniejszych błędów. Wartość z tego miejsca to próg błędu rekonstrukcji (*threshold*).

Gdy do sieci trafia nowy pakiet ze zbioru testowego, model przetwarza go i wylicza błąd rekonstrukcji. Jeśli ten błąd jest większy niż ustalony próg, model zgłasza alarm, w przeciwnym przypadku uznaje to za normę.

Dane sieciowe charakteryzują się występowaniem ekstremalnych wartości odstających, np. gigantyczne transfery plików vs. małe pakiety ping. Z tego powodu do skalowania i przetwarzania danych wykorzystano algorytm *RobustScaler* [13]. Jest to algorytm normalizacji danych, który zamiast średniej i odchylenia standardowego wykorzystuje medianę oraz rozstęp ćwiartkowy, co pozwala zachować dynamikę "typowego" ruchu sieciowego. Dodatkowo wykorzystano transformację logarytmiczną, celem zmniejszenia rzędu wielkości danych, co ułatwia sieci neuronowej zbieżność (konwergencję) i zapobiega dominacji cech wolumetrycznych nad innymi.

Kluczowe przy współpracy z *RobustScaler*, który generuje ujemne wartości dla połowy danych, jest zastosowanie funkcji aktywacji *LeakyReLU* [14]. Jest to wariant funkcji *ReLU*, który dla wartości ujemnych nie zeruje sygnału całkowicie, lecz przepuszcza go z niewielkim mnożnikiem (np. 0.1). *LeakyReLU* zapobiega problemowi "martwych neuronów" i pozwala sieci uczyć się różnic również w pakietach o wartościach poniżej mediany.

Zastosowaną funkcją straty jest *MSELoss* [15]. Jest to funkcja celu obliczająca średnią kwadratów różnic pomiędzy wartościami wejściowymi a zrekonstruowanymi przez *Autoencoder*. Dzięki podnoszeniu różnic do kwadratu, funkcja ta nieproporcjonalnie mocno karze duże błędy, będąc jednocześnie tolerancyjną dla małych odchylen. W kontekście cyberbezpieczeństwa oznacza to, że pakiet ataku, który drastycznie odbiega od normy, generuje bardzo wysoki wynik błędu, co ułatwia jego odseparowanie od tła normalnego ruchu sieciowego.

Eksperymenty przeprowadzono wykorzystując procedurę k -krotnej walidacji krzyżowej (*k-fold cross-validation*). Metoda ta polega na losowym podziale całego dostępnego zbioru danych na k rozłącznych podzbiorów (tzw. *foldów*) o równej liczności. Proces ewaluacji powtarzany jest k -krotnie: w każdej iteracji jeden z podzbiorów pełni rolę zbioru testowego, podczas gdy pozostałe $k - 1$ podzbiorów stanowi zbiór treningowy. Zastosowanie tego podejścia podyktowane było koniecznością uzyskania wiarygodnej i stabilnej oceny skuteczności modelu, niezależnej od specyficznego podziału danych na część uczącą i weryfikującą. Walidacja krzyżowa eliminuje ten problem, gwarantując, że każda próbka ze zbioru danych zostanie dokładnie raz wykorzystana do testowania i $k - 1$ razy do treningu, a ostateczny wynik jest średnią arytmetyczną ze wszystkich prób. W ramach badań przyjęto parametr podziału k równy 5, a w każdej iteracji model był trenowany przez 5 epok [17].

Zaimplementowany *Autoencoder* został porównany z modelami *Isolation Forest* oraz *OC SVM* na zbiorze danych KDD99.

Isolation Forest to algorytm uczenia nienadzorowanego, który opiera się na założeniu, że anomalie są zdarzeniami rzadkimi i znacząco różniącymi się od reszty populacji.

Algorytm konstruuje zespół losowych drzew binarnych, w których rekurencyjnie dzieli dane za pomocą przypadkowo dobieganych atrybutów i punktów podziału. W tak powstałej strukturze ataki zostają odseparowane od reszty zbioru bardzo wcześnie, łądząc na płytszych poziomach drzewa, podczas gdy gęste skupiska ruchu normalnego wymagają znacznie większej liczby podziałów. Ostateczna ocena anomalii jest zatem wyliczana na podstawie średniej długości ścieżki potrzebnej do wyizolowania danego punktu – im jest ona krótsza, tym wyższe prawdopodobieństwo, że badana próbka stanowi zagrożenie [14].

One-Class Support Vector Machine to algorytm uczenia nienadzorowanego służący do detekcji nowości (novelty detection), który modyfikuje klasyczne podejście SVM, rezygnując z poszukiwania granicy między dwiema klasami na rzecz wyznaczenia ścisłej otoczki wokół danych reprezentujących ruch normalny. Mechanizm ten wykorzystuje tzw. *kernel trick* do rzutowania danych wejściowych do wysoko-wymiarowej przestrzeni cech, gdzie algorytm stara się znaleźć optymalną hiperpłaszczyznę oddzielającą skupisko danych normalnych od początku układu współrzędnych z maksymalnym możliwym marginesem. W rezultacie powstaje nieliniowa granica decyzyjna, która definiuje obszar "normalności". Wszelkie nowe obserwacje, które po transformacji znajdują się po zewnętrznej stronie tej granicy (czyli bliżej początku układu współrzędnych), są klasyfikowane jako anomalie [15]. W niniejszym badaniu, ze względu na wysoką złożoność obliczeniową klasycznej implementacji (rzędu $O(n^3)$) oraz duży wolumen danych sieciowych, zastosowano wariant zoptymalizowany wykorzystujący metodę Stochastycznego Spadku wzdłuż Gradientu (SGD). Pozwoliło to na zredukowanie złożoności do poziomu liniowego $O(n)$, umożliwiając efektywne przetwarzanie zbiorów zawierających miliony próbek [16].

IV. ANALIZA ZBIORÓW DANYCH

Ewaluację modelu przeprowadzono w oparciu o trzy różniące zbiory danych, reprezentujące odmienne charakterystyki ruchu sieciowego oraz formaty zapisu cech. Pierwszym z nich jest zbiór *KDD Cup 1999 Data* [12], powszechnie stosowany do porównywania skuteczności systemów IDS. Drugim źródłem danych był zestaw oparty na formacie *NetFlow v9* [13] Trzeci zbiór, *CoReS-IoT*, koncentruje się na specyfice Internetu Rzeczy i wyróżnia się strukturą etykietowania – oferuje on jedynie binarną klasyfikację (norma/atak) w ostatniej kolumnie. Taki dobór danych wejściowych pozwolił na wszechstronną weryfikację zdolności generalizacji modelu oraz dokładną ocenę jego skuteczności. Liczność danych:

- *KDD Cup 1999 Data* - 1 074 992 próbek. Liczba cech: 39
- *Netflow V9* - 1 000 000 próbek. Liczba cech: 19
- *cores-iot* - 1 008 748 próbek. Liczba cech: 19

A. Zbiór KDD

Na rysunku 1 przedstawiono transformację rozkładu cechy *src_bytes* w zbiorze KDD, która oznacza liczbę bajtów

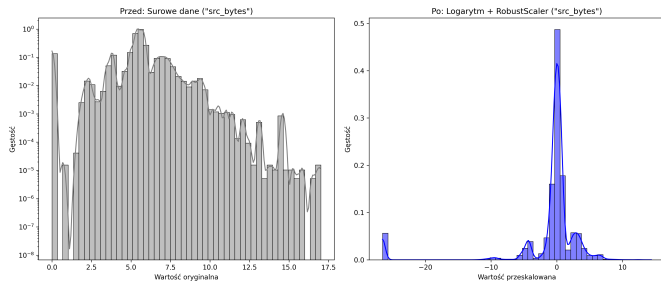


Fig. 1. Wartości cechy *src_bytes* przed i po przeskalowaniu

źródłowych. Po lewej stronie widzimy "poszarpany" rozkład, gdzie dane nie mają wyraźnego centrum. Dla sieci neuronowej taki rozkład jest trudny do interpretacji, ponieważ wagi musiałyby adaptować się do bardzo szerokiego zakresu wartości, co grozi niestabilnością treningu. Zastosowanie *RobustScalera*, który do skalowania wykorzystuje odporną na wartości skrajne medianę, pozwoliło na przekształcenie silnie skośnego rozkładu danych sieciowych w rozkład zbliżony do normalnego, wycentrowany w zerze. Jest to kluczowe dla stabilności procesu uczenia głębokich sieci neuronowych i zapobiegania dominacji modelu przez wartości skrajne.

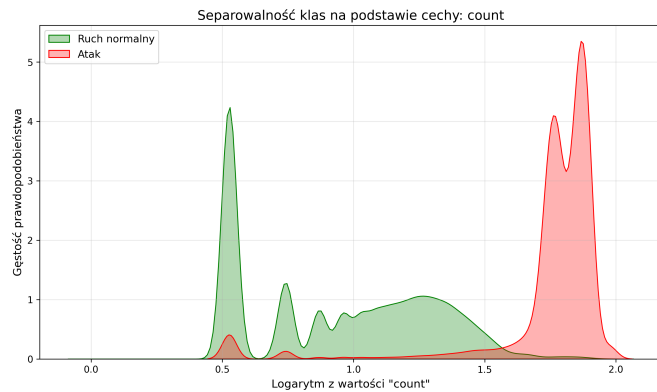


Fig. 2. Separowalność klas na podstawie cechy *count*

Wykres na rysunku 2 przedstawia analizę gęstości prawdopodobieństwa dla cechy *count* po transformacji logarytmicznej. Cecha ta oznacza liczbę połączeń do tego samego hosta docelowego w ciągu ostatnich 2 sekund. Wykres pokazuje bardzo wyraźny podział między ruchem normalnym a atakiem, co oznacza, że cecha *count* może być kluczowym predyktorem w procesie detekcji anomalii.

B. Zbiór netflow

Na rysunku 3 przedstawiono transformację rozkładu cechy *in_bytes* w zbiorze netflow, który analogicznie jak w zbiorze kdd, oznacza liczbę bajtów źródłowych. Ponownie można zauważyć, że dane po lewej stronie nie tworzą rozkładu normalnego, a mają wiele lokalnych szczytów. Zastosowanie *RobustScalera* uwypukliło istnienie różnych klas ruchu (widoczne jako odseparowane "górki" na prawym

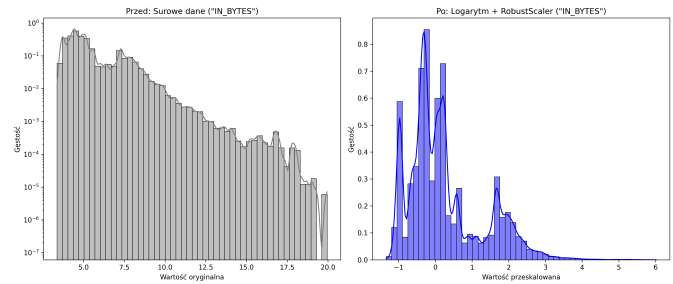


Fig. 3. Wartości cechy *IN_BYTES* przed i po przeskalowaniu

wykresie), które na lewym wykresie były rozmyte przez różnice skali. Sprowadzenie danych do wąskiego zakresu wokół zera pozwoliło osiągnąć stabilność procesu uczenia i uniknąć zdominowania modelu przez skrajne wartości.

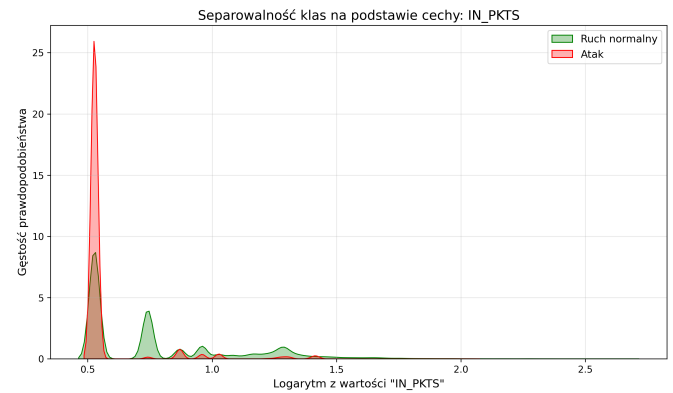


Fig. 4. Separowalność klas na podstawie cechy *IN_PKTS*

Wykres na rysunku 4 przedstawia analizę gęstości prawdopodobieństwa dla cechy *in_pkts* po transformacji logarytmicznej. Widzimy, że zarówno ruch normalny, jak i ataki mają swoje maksimum w wartości 0.5–0.6 na osi logarytmicznej. Oznacza to, że w przeciwieństwie do cechy *count* w zbiorze KDD, dla tej konkretnej cechy, nie ma wyraźnej granicy między atakiem a normą. Prowadzi to do wniosku, że niektóre parametry mogą stanowić silny, samodzielny predyktor anomalii, natomiast inne wykazują znaczne współdziałanie zakresów wartości dla ruchu normalnego i ataków. Podkreśla to konieczność stosowania modeli wielowymiarowych, takich jak *Autoencoder*, które zamiast polegać na pojedynczych wskaźnikach, uczą się złożonych, nieliniowych korelacji pomiędzy wieloma cechami jednocześnie, co pozwala na skuteczną klasyfikację nawet w przypadku niejednoznaczności pojedynczych atrybutów.

V. WYNIKI

A. Analiza zmiany wartości współczynników w zależności od ustalonego progu odcięcia

Tabela I przedstawia wyniki badań modelu na zbiorze KDD. Pogrubieniem zaznaczono wiersz z percentylem (p),

TABLE I
WYNIKI BADAŃ MODELU NA ZBIORZE DANYCH KDD

p	Czułość	Precyzja	F1 Score	F1 +/-	FP	TP
60	1.000000	0.446460	0.617313	0.000878	65012	52435
65	1.000000	0.479588	0.648272	0.000714	56899	52435
70	0.999992	0.518116	0.682575	0.000839	48768	52435
75	0.999977	0.563196	0.720563	0.000965	40667	52434
80	0.999954	0.617349	0.763395	0.000592	32499	52433
85	0.999886	0.682929	0.811558	0.000454	24342	52429
90	0.999661	0.763585	0.865819	0.000271	16229	52417
95	0.998600	0.865867	0.927508	0.000779	8111	52362
96	0.997662	0.889547	0.940507	0.000904	6495	52313
97	0.996254	0.914804	0.953793	0.001555	4865	52239
98	0.992364	0.941114	0.966058	0.001947	3255	52035
99	0.896319	0.964537	0.919813	0.113571	1631	46999

dla którego model uzyskał najwyższy *F1-Score*. Kolumna *FP* przedstawia liczbę fałszywych alarmów *False-Positive*, a kolumna *TP* przedstawia liczbę wykrytych ataków *True-Positive* (wartości uśrednione z 5 testów).

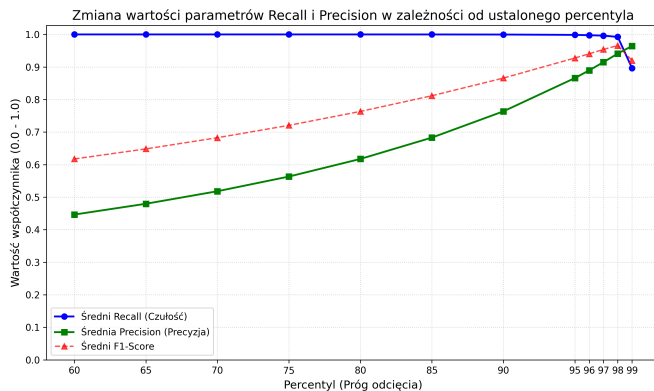


Fig. 5. Analiza współczynników dla różnych progów odcięcia testowanego modelu na zbiorze danych KDD 99

Na rysunku 5 zostały przedstawione wyniki dla pierwszego zbioru danych (KDD99). Wykres przedstawia zależność pomiędzy współczynnikami *Recall*, *Precision* i *F1-Score* w zależności od ustalonego percentyla (progu odcięcia). Można zaobserwować, że dla nisko ustalonego progu (percentyl = 60) model charakteryzuje się idealną czułością (wykrywa wszystkie ataki), ale odbywa się to kosztem niskiej precyzji, co powoduje dużą ilość fałszywych alarmów. Im próg odcięcia ustawiony jest dalej, tym czułość modelu spada, ale rośnie jego precyzja. W konsekwencji spada liczba fałszywych alarmów, ale nie są wykrywane wszystkie ataki. Najlepszą wartość *F1* na tym zbiorze model osiągnął dla percentyla 98.

Tabela II przedstawia wyniki badań modelu na zbiorze Netflow V9. Pogrubieniem zaznaczono wiersz z percentylem (p), dla którego model uzyskał najwyższy *F1-Score*. Kolumna *FP* przedstawia liczbę fałszywych alarmów *False-Positive*, a kolumna *TP* przedstawia liczbę wykrytych ataków *True-Positive* (wartości uśrednione z 5 testów).

Na rysunku 6 zostały przedstawione wyniki dla drugiego zbioru danych (Netflow V9). Wykres przedstawia zależność pomiędzy współczynnikami *Recall*, *Precision* i *F1-Score* w zależności od ustalonego percentyla (progu odcięcia). Można

TABLE II
WYNIKI BADAŃ MODELU NA ZBIORZE DANYCH NETFLOW

p	Czułość	Precyzja	F1 Score	F1 +/-	FP	TP
60	0.987009	0.274127	0.429082	0.000715	69337	26182
65	0.978716	0.299719	0.458903	0.003965	60659	25962
70	0.958661	0.328411	0.489224	0.006141	52002	25430
75	0.934158	0.363445	0.523291	0.005729	43399	24780
80	0.926219	0.414516	0.572711	0.005947	34701	24569
85	0.921710	0.484948	0.635515	0.005253	25966	24450
90	0.916930	0.583436	0.713110	0.005717	17365	24323
95	0.794217	0.699952	0.740376	0.107097	8701	21068
96	0.531964	0.615179	0.551157	0.243378	6935	14111
97	0.155683	0.399146	0.221336	0.127846	5183	4129
98	0.008030	0.055692	0.014033	0.014764	3441	213
99	0.000023	0.000346	0.000042	0.000057	1743	1

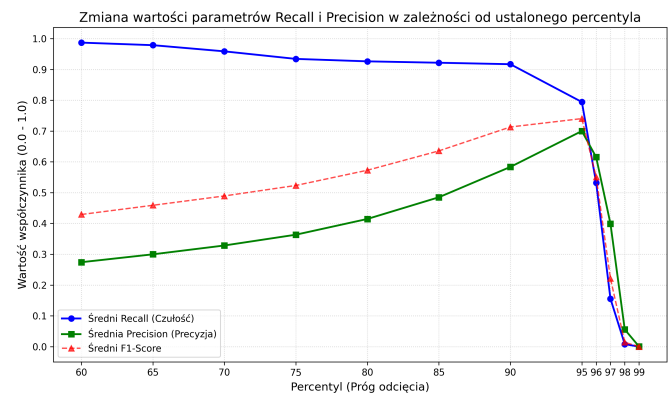


Fig. 6. Analiza współczynników dla różnych progów odcięcia testowanego modelu na zbiorze danych Netflow V9

zaobserwować, że podobnie jak dla zbioru danych KDD 99, dla nisko ustalonego percentyla model wykrywa prawie 100% ataków, ale odbywa się to kosztem wysokiej liczby fałszywych alarmów. Najwyższą wartość współczynnika *F1-Score* model uzyskał dla percentyla 95. Im większy percentyl, tym czułość modelu zaczyna gwałtownie spadać. Przez zbyt nisko ustalony próg błędów rekonstrukcji, model zaczyna tracić zdolność wykrywania ataków.

TABLE III
WYNIKI BADAŃ MODELU NA ZBIORZE DANYCH CORESIOT

p	Czułość	Precyzja	F1 Score	F1 +/-	FP	TP
60	0.999891	0.741815	0.851733	0.000467	37548	107882
65	0.999761	0.766600	0.867791	0.000176	32842	107868
70	0.999320	0.792965	0.884263	0.000505	28151	107821
75	0.996610	0.820955	0.900289	0.002877	23450	107528
80	0.984196	0.849843	0.912030	0.010084	18756	106189
85	0.868424	0.869521	0.868528	0.025277	14028	93698
90	0.478356	0.845908	0.610974	0.016340	9390	51612
95	0.108712	0.703220	0.187780	0.041165	4702	11729
96	0.068163	0.652815	0.123238	0.026202	3760	7354
97	0.046950	0.634370	0.087340	0.017598	2829	5065
98	0.034886	0.660114	0.066232	0.011404	1893	3764
99	0.026148	0.748868	0.050524	0.004622	939	2821

Tabela III przedstawia wyniki badań modelu na zbiorze cores-iot. Pogrubieniem zaznaczono wiersz z percentylem (p), dla którego model uzyskał najwyższy *F1-Score*. Kolumna *FP* przedstawia liczbę fałszywych alarmów *False-Positive*,

a kolumna *TP* przedstawia liczbę wykrytych ataków *True-Positive* (wartości uśrednione z 5 testów).

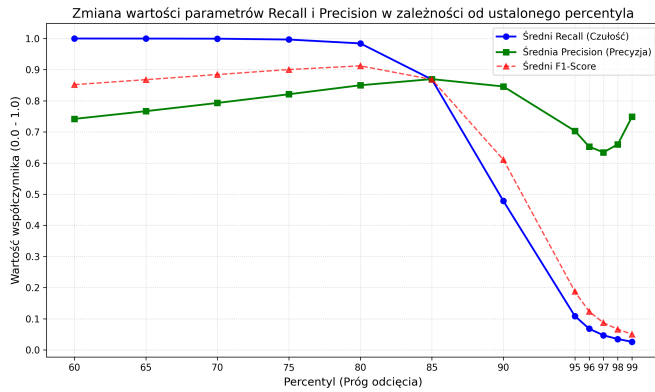


Fig. 7. Analiza współczynników dla różnych progów odcięcia testowanego modelu na zbiorze danych cores-iot

Na rysunku 7 zostały przedstawione wyniki dla trzeciego zbioru danych (cores-iot). Wykres przedstawia zależność pomiędzy współczynnikami *Recall*, *Precision* i *F1-Score* w zależności od ustalonego procentyla (progu odcięcia). Podobnie jak w przypadku zbioru Netflow V9, model od pewnego momentu (w tym przypadku dla procentyla 80) zaczyna gwałtownie tracić zdolność skutecznego wykrywania ataków sieciowych. Dla niższych procentyli (wyższy próg błędów rekonstrukcji) model jest w stanie wykrywać nawet ponad 99% ataków z tego zbioru.

B. Histogramy błędów rekonstrukcji

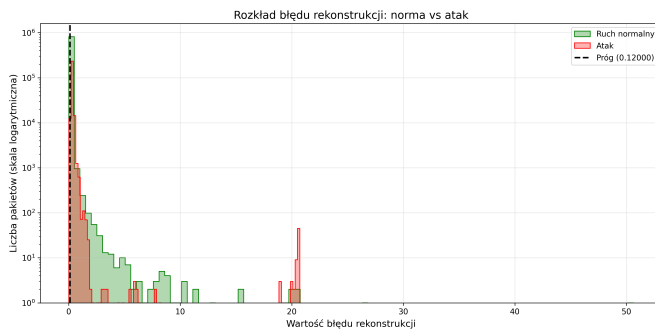


Fig. 8. Histogram błędów rekonstrukcji dla wyników testowanego modelu na zbiorze danych KDD 99

Na rysunku 8 przedstawiono wykres rozkładu błędów rekonstrukcji dla ruchu normalnego oraz ataków (anomalii) dla zbioru KDD99. Można zauważyć, że zarówno ruch normalny, jak i większość ataków generują błąd rekonstrukcji w przedziale 0.0 - 2.0, ale czerwony wykres jest przesunięty minimalnie na prawo od zielonego. Jest zatem możliwe skuteczne separowanie ruchu normalnego do ataków dla tego zbioru danych, lecz wymaga to ustawienia bardzo niskiego błędów rekonstrukcji (0.12). Dzięki temu możliwe jest wykrycie

znacznej większości ataków, przy stosunkowo niewielkiej liczbie fałszywych alarmów (tabela II).

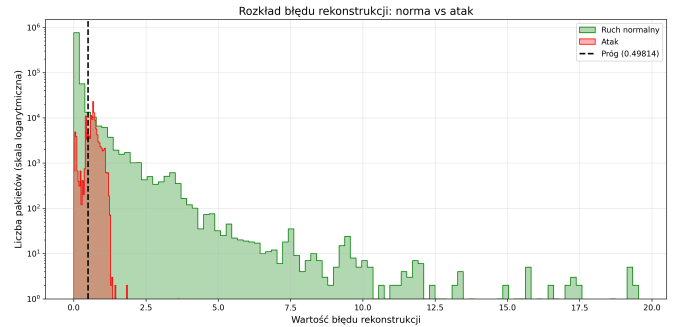


Fig. 9. Histogram błędów dla wyników testowanego modelu na zbiorze danych Netflow V9

Na rysunku 9 przedstawiono wykres rozkładu błędów rekonstrukcji dla ruchu normalnego oraz ataków (anomalii) dla zbioru Netflow V9. Można na nim zaobserwować, że zbiór ten jest trudniejszy do analizy dla zaimplementowanego *Autoencoder*, ponieważ grupa ataków okazała się posiadać niski błąd rekonstrukcji (poniżej 2.0), a spora część ruchu normalnego posiada błąd rekonstrukcji osiągający wartość równą niemal 20.0. Ataki okazały się zatem bardzo podobne do części normy, którą model przyswoił najlepiej, a w ruchu normalnym istnieje podgrupa pakietów, które są bardzo skomplikowane lub rzadkie. Mimo to dla ustalonego progu model jest w stanie wykrywać ponad 90% ataków, ale odbywa się to kosztem większej liczby fałszywych alarmów (tabela II).

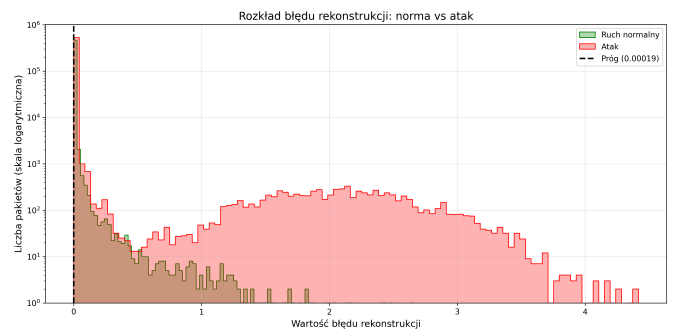


Fig. 10. Histogram błędów dla wyników testowanego modelu na zbiorze danych cores-iot

Na rysunku 10 przedstawiono wykres rozkładu błędów rekonstrukcji dla ruchu normalnego oraz ataków (anomalii) dla zbioru cores-iot. W tym zbiorze znaczna część ataków okazała się posiadać wyraźnie wyższy błąd rekonstrukcji niż ruch normalny. Jednakże, duża ich część posiada błąd rekonstrukcji bardzo zbliżony do ruchu normalnego. Dzięki niskiemu ustalonemu progowi błędów (0.000019), *Autoencoder* jest w stanie wykryć na tym zbiorze ponad 98% ataków, przy minimalnej liczbie fałszywych alarmów (tabela III).

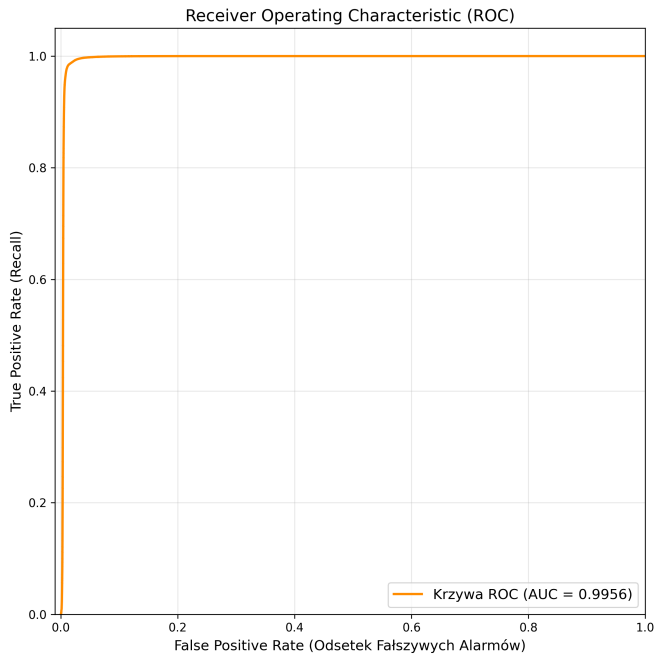


Fig. 11. Krzywa ROC dla wyników testowanego modelu na zbiorze danych KDD 99

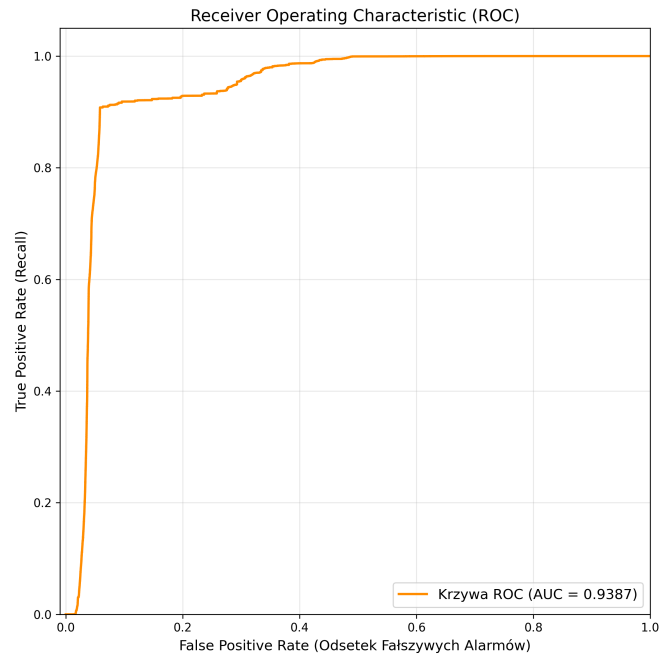


Fig. 12. Krzywa ROC dla wyników testowanego modelu na zbiorze danych cores-iot

C. Krzywe ROC

Krzywa ROC przedstawiona na rysunku 11 dla wyników modelu testowanego na zestawie danych KDD99 charakteryzuje się niemal idealnym przebiegiem, osiągając wartość $AUC = 0.9956$. Oznacza to bardzo wysoką zdolność modelu do rozróżniania danych pomiędzy ruchem normalnym a atakami. Już przy bardzo niskim odsetku fałszywych alarmów (FPR) model osiąga maksymalną wykrywalność ($TPR = 1.0$).

Krzywa ROC przedstawiona na rysunku 12 dla wyników modelu testowanego na zestawie danych Netflow V9 wartość $AUC = 0.9387$. Oznacza to wysoką zdolność modelu do rozróżniania danych pomiędzy ruchem normalnym a atakami, aczkolwiek jest to wynik gorszy, niż dla zbioru danych KDD 99. Można też zauważyć, że by móc wykrywać na tym zbiorze 100% ataków, trzeba się liczyć z większą liczbą fałszywych alarmów.

Krzywa ROC przedstawiona na rysunku 13 dla wyników modelu testowanego na zestawie danych cores-iot osiąga wartość $AUC = 0.8956$. Dla tego zbioru model też osiągnął zatem wysoką skuteczność w rozróżnianiu ataków od normalnego ruchu sieciowego, ale jest to wynik gorszy, niż dla dwóch poprzednich zbiorów danych.

VI. PORÓWNANIE Z INNYMI MODELAMI

Na rysunku 14 przedstawiono wykres porównujący wartość współczynnika czułości między modelami *Autoencoder*, *Isolation Forest*, a *OC SVM* dla zbioru danych KDD w zależności od przyjętego progu odcięcia. *Autoencoder* i *Isolation Forest* wykazują tutaj bardzo zbliżoną, wysoką wydajność. Oba modele utrzymują Recall na poziomie bliskim 100% aż do 95. percentyla. Są więc w stanie wykryć niemal wszystkie ataki w

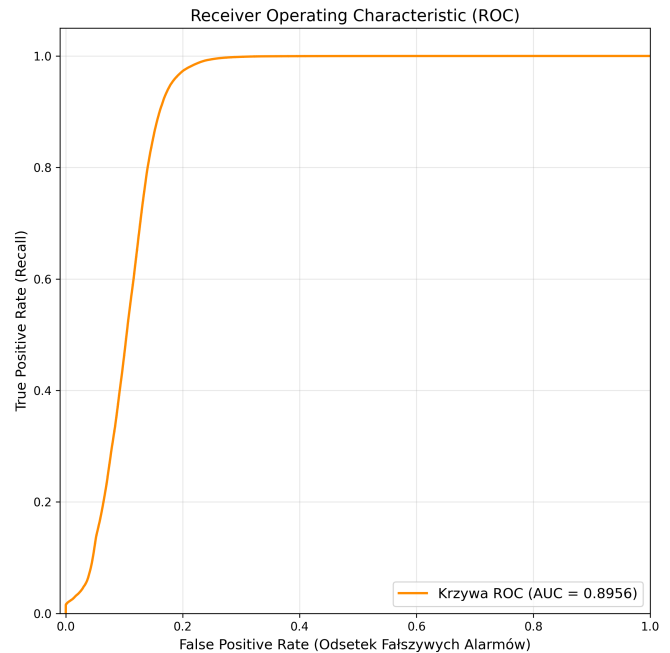


Fig. 13. Krzywa ROC dla wyników testowanego modelu na zbiorze danych cores-iot

tym zbiorze danych, nawet przy dość restrykcyjnych progach. Dopiero przy bardzo wysokich percentylach (98-99), czułość zaczyna delikatnie spadać. Co ciekawe, *Isolation Forest* radzi sobie tu minimalnie lepiej, utrzymując wyższą czułość na samym końcu skali. Natomiast model *OC SVM* wykazuje krytyczną niestabilność powyżej 90. percentyla, gdzie Recall

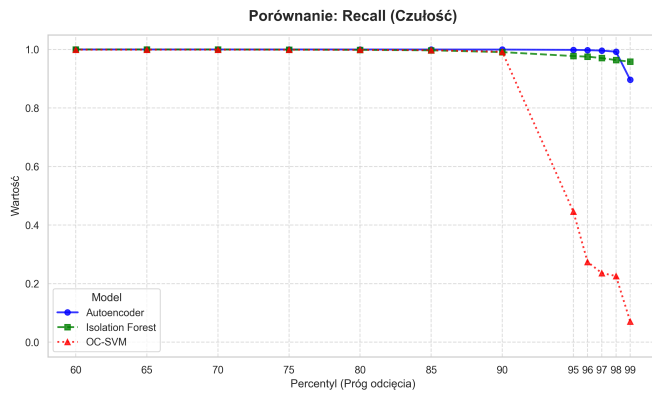


Fig. 14. Porównanie współczynnika czułości między modelami

spada do poziomu ok. 40%, a dla 99. percentyla osiąga poziom bliski zeru. Oznacza to, że funkcja decyzyjna dla tego modelu słabo koreluje z rzeczywistym charakterem ataków w obszarze wartości skrajnych.

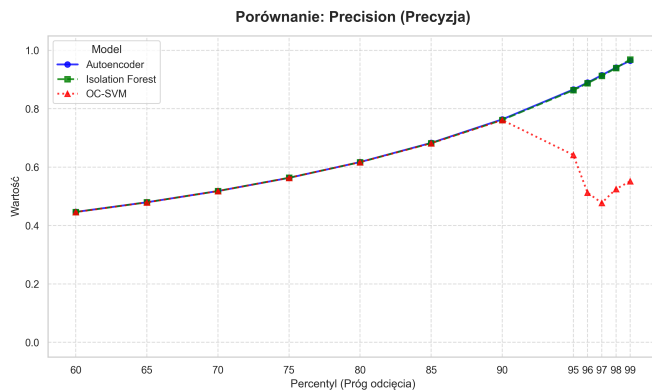


Fig. 15. Porównanie współczynnika precyzji między modelami

Na rysunku 15 przedstawiono wykres porównujący wartość współczynnika precyzji między modelami *Autoencoder*, *Isolation Forest*, a *OC SVM* dla zbioru danych KDD w zależności od przyjętego progu odcięcia. Można na nim zauważyć niemal identyczne zachowanie modeli *Autoencoder* oraz *Isolation Forest*, podobnie jak w przypadku analizy współczynnika czułości. Oba modele wykazują idealną, rosnącą charakterystykę, osiągając wartość bliską 1.0 przy 99. percentylu. Oznacza to, że oba algorytmy poprawnie wykrywają anomalie. Im bardziej rygorystyczny próg ustawimy, tym większa szansa, że zgłoszony alarm jest faktycznym atakiem, a nie fałszywym alarmem. Natomiast model *OC SVM* do 90. percentyla osiąga wyniki bardzo zbliżone, do pozostałych dwóch modeli, natomiast później następuje drastyczne załamanie. Oznacza to, że w grupie próbek, które *OC SVM* uznaje za "najbardziej ekstremalne anomalie" (górne 5% wyników funkcji decyzyjnej), znajduje się bardzo dużo ruchu normalnego.

Na rysunku 16 przedstawiono wykres porównujący wartość współczynnika F1 między modelami *Autoencoder*, *Isolation Forest*, a *OC SVM* dla zbioru danych KDD w zależności

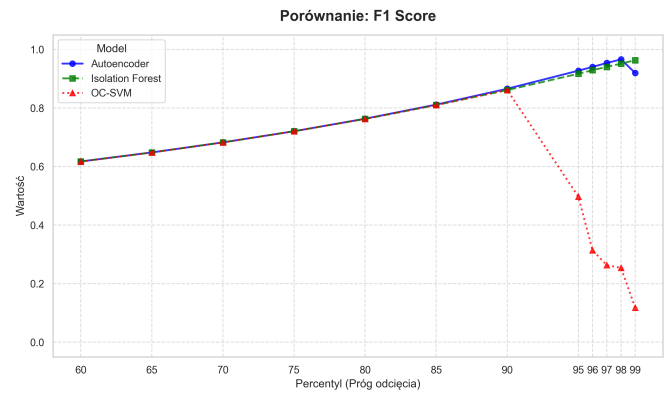


Fig. 16. Porównanie współczynnika F1 między modelami

od przyjętego progu odcięcia. Potwierdza on trend widoczny na wykresach 14 i 15. *Autoencoder* i *Isolation Forest* osiągają przez większość zakresu identyczne wartości. Wartość współczynnika F1 rośnie wraz z zaostreniem progu od 60. do 98. percentyla. Wynika to z faktu, że przy niższych progach modele generują dużo fałszywych alarmów (niska precyzja), co zaniża wynik F1. Podnoszenie progu drastycznie poprawia precyzję przy zachowaniu wysokiej czułości. Oba modele osiągają wynik maksymalny powyżej 0.95. Świadczy to o tym, że dla tego zbioru danych oba podejścia są bardzo skuteczne. Warto też podkreślić, że w przeciwieństwie do *Autoencodera*, *Isolation Forest* nie notuje spadku przy 99. percentylu, lecz utrzymuje, a nawet lekko poprawia swój wynik. Sugeruje to, że ranking anomalii tworzony przez *Isolation Forest* jest nieco "czystszy" na samym szczycie, podczas gdy *Autoencoder* ma tam już pewną część trudnego do rozpoznania ruchu normalnego lub gubi część ataków. Wykres ten również pokazuje słabość modelu *OC SVM*. Powyżej 90. percentyla następuje gwałtowny spadek wartości współczynnika F1 z poziomu ok. 0.85 do ok. 0.10. Oznacza to, że model ten jest niezdolny do pracy w reżimie wysokiej precyzji. Próba zminimalizowania fałszywych alarmów kończy się całkowitą utratą zdolności detekcji ataków.

VII. WNIOSKI

W odniesieniu do pierwszego pytania badawczego (*Q1*), przeprowadzone badania potwierdziły, że architektura *Autoencodera* jest skutecznym narzędziem do detekcji anomalii w ruchu sieciowym. Osiągnięty wynik pola pod krzywą ROC (w zależności od zbioru między 0.8977 a 0.9975) wskazuje na wysoką zdolność modelu do generalizacji i odróżniania ruchu normalnego od ataków. Model skutecznie nauczył się kompresować i rekonstruować charakterystykę ruchu bezpiecznego, traktując ataki jako anomalie.

Kluczowym elementem sukcesu okazał się dobór odpowiedniej metody przetwarzania danych. Surowe dane sieciowe charakteryzują się rozkładem potęgowym, co utrudnia zbieżność sieci neuronowych. Zastosowanie transformacji logarytmicznej w połączeniu z *RobustScaler*

pozwoili na skuteczn separacj klas.

Badania ujawnily istotny kompromis pomiędzy czuoloscia a precyzja systemu. Wykresy gesci bledu rekonstrukcji pokazaly, ze w zaleznosci od zbioru danych, rozroznianie ataku od normalnego ruchu sieciowego moze byc dla modelu wyzwaniem. Przyjeta strategia ustalania progu odciecia (threshold) priorytetyzowala wysoki poziom wykrywalnosci atakow (jak najwieksza precyzja), co jest podejsciem pozadanym w cyberbezpiecienstwie. Odbulo sie to jednak kosztem wzrostu liczby falszywych alarmow (False Positive) w obszarze, gdzie zlony ruch normalny matematycznie przypomina anomalie.

Analiza porownawcza wykazala, ze proponowany *Autoencoder* osiaga wydajnosc zblizona do algorytmu *Isolation Forest*, utrzymujac wysoki poziom metryki F1-score. W przeciwienstwie do modelu *One-Class SVM*, ktory wykazal znaczna niestabilnosc i gwaltowny spadek czuosci przy restrykcyjnych ustawieniach, siec neuronowa zachowala skutecnosc detekcji nawet przy bardzo wysokich wymaganiach co do precyzji. Potwierdza to, ze podejscie oparte na rekonstrukcji danych stanowi konkurencyjna i stabilna alternatywe dla klasycznych metod uczenia maszynowego.

Odpowiedz na drugie pytanie badawcze (Q2) jest twierdza. Zastosowana strategia uczenia metoda *open-set recognition* (trening wylicznie na danych normalnych) pozwolila modelowi na traktowanie kazdego typu ataku – niezaleznie od tego, czy byl on znany w momencie treningu, czy tez jest nowym zagrozeniem typu Zero-Day - jako anomalii.

Zrealizowany projekt dowodzi, ze *Autoencoder* moze stanowic solidny fundament nowoczesnych systemow wykrywania atakow sieciowych. Jednakze, w przypadku wyrafinowanych zagrozen, ktore precyzyjnie nasladuja charakterystyce ruchu normalnego, samodzielne wykorzystanie tej architektury moze okazac sie niewystarczajace i wymagac wsparcia dodatkowymi mechanizmami weryfikacji.

A. Deklaracja wykorzystania narzedzi AI

1) Google Gemini:

- Zastosowanie: Model jzykowy LLM
- Zakres: Wsparcie w pisaniu skryptow do wczytywania i obróbki danych oraz generowania wykresow. Redagowanie i korekta tekstu.
- Uwagi: Wszystkie wygenerowane tresci zostaly poddane weryfikacji i ostatecznej edycji przez autorow pracy.

REFERENCES

- [1] <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [2] <https://www.kaggle.com/datasets/ashtcoder/network-data-schema-in-the-netflow-v9-format>
- [3] <https://docs.python.org/3/>
- [4] <https://scikit-learn.org/stable/>
- [5] <https://docs.pytorch.org/docs/stable/index.html>
- [6] <https://pandas.pydata.org/docs/>
- [7] <https://numpy.org/doc/2.3/>
- [8] <https://matplotlib.org/stable/index.html>
- [9] <https://colab.research.google.com/>
- [10] Mahmoud Said Elsayed, Nhen-An Le-Khac, Soumyabrata Dev, and Anca Delia Jurcut. 2020. Network Anomaly Detection Using LSTM Based Autoencoder. In Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks (Q2SWinet '20). Association for Computing Machinery, New York, NY, USA, 37–45. <https://doi.org/10.1145/3416013.3426457>
- [11] Geng, Chuanxing, Sheng-jun Huang, and Songcan Chen. "Recent advances in open set recognition: A survey." IEEE transactions on pattern analysis and machine intelligence 43.10 (2020): 3614-3631.
- [12] KDD Cup 1999 Data, <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [13] Network data schema in the Netflow V9 format, <https://www.kaggle.com/datasets/ashtcoder/network-data-schema-in-the-netflow-v9-format>
- [14] Lesouple, J., Baudoin, C., Spigai, M., & Tourneret, J. Y. (2021). Generalized isolation forest for anomaly detection. Pattern Recognition Letters, 149, 109-119.
- [15] Hejazi, Maryamsadat, and Yashwant Prasad Singh. "One-class support vector machines approach to anomaly detection." Applied Artificial Intelligence 27.5 (2013): 351-366.
- [16] Shieh, Chin-Shiuh, et al. "Detection of unknown DDoS attack using reconstruct error and one-class SVM featuring stochastic gradient descent." Mathematics 11.1 (2022): 108.
- [17] Anguita, Davide, et al. "The 'K' in K-fold Cross Validation." Esann. Vol. 102. 2012.